CHAPTER ONE

INTRODUCTION

1.1 Introduction

The way businesses acquire and use software has shifted considerably over the past two decades. Where enterprise systems were once the exclusive territory of large corporations with the capital to purchase licences, deploy servers, and employ dedicated IT staff, the rise of cloud computing has steadily brought sophisticated software tools within reach of much smaller organisations. Central to this shift is the Software-as-a-Service (SaaS) delivery model, through which applications are hosted by a provider and accessed by users over the internet, typically on a subscription basis. Businesses no longer need to manage their own infrastructure; they simply log in and work.

Inventory management is one area where this shift has made a measurable difference. For any business that buys, holds, or sells physical goods, knowing what is in stock, where it is, and how fast it is moving is not a luxury, it is a basic operational requirement. Getting that information wrong is expensive. Stockouts mean lost sales and frustrated customers. Overstocking ties up working capital in goods sitting idle on shelves. Poor record-keeping leads to discrepancies that take significant time to resolve. These are problems that competent inventory software is designed to prevent, yet many small and medium-sized businesses (SMEs) in Nigeria still manage their stock with handwritten ledgers, spreadsheets, or basic desktop applications that cannot be accessed remotely and provide no real-time visibility.

The reasons for this gap are not hard to identify. Most of the well-regarded SaaS inventory platforms available today. Tools such as Zoho Inventory, Cin7, or Linnworks are priced for markets in Europe and North America, where internet costs are low and card payment infrastructure is mature. Beyond the pricing issue, these platforms are typically deployed as single-tenant systems: each customer gets their own separate installation of the software, its own server, and its own database. For a vendor trying to serve hundreds of small Nigerian businesses, this model is operationally expensive. Each new customer adds another unit of infrastructure to manage, and those costs get passed back down the chain, reinforcing the affordability problem.

Multi-tenancy offers a way around this. Under a multi-tenant architecture, a single deployment of the software serves many customers simultaneously. Each customer referred to as a tenant sees only their own data, but the underlying application and infrastructure are shared. Kavis (2014) describes multi-tenancy as one of the defining characteristics of true SaaS design, as opposed to simple hosted software where each customer is just running their own private copy. When the infrastructure costs are shared across dozens or hundreds of tenants, the per-tenant cost drops significantly, making it possible for a vendor to offer the platform at a price point that Nigerian SMEs can realistically afford.

The second major architectural choice in this project is microservices. Rather than building the inventory platform as a single, large application commonly called a monolith the system will be decomposed into a set of small, independently deployable services. One service will handle authentication and tenant management. Another will manage inventory records. A third will process orders and sales. A fourth will send notifications. Each service will have its own codebase and its own deployment pipeline, and each can be scaled or updated without

touching the others. Newman (2021) describes this arrangement as microservices architecture, noting that the key benefit is that individual parts of the system can evolve and scale independently, reducing the risk and the downtime that come with releasing changes to a tightly coupled monolith.

This project brings these two architectural approaches together to produce a working multi-tenant SaaS inventory management platform. The platform is intended to serve the practical needs of Nigerian SMEs while demonstrating the engineering principles that make it work. The chapters that follow document the problem in more detail, the design choices made, the implementation process, and the results of testing the system against specific performance and security criteria.

1.2 Background of the Study

Inventory management is not a new discipline. Long before computers existed, merchants kept physical ledgers recording what goods they held, what they had sold, and what they needed to reorder. The principles have remained broadly the same: know what you have, know what you need, and keep the two in balance. What has changed is the scale and speed at which modern businesses operate. A retailer managing thousands of stock-keeping units across several locations cannot manage that volume reliably on paper. The information moves too fast and the consequences of error are too costly.

The computerisation of inventory management began in earnest in large manufacturing organisations in the late 1960s and 1970s, driven by the development of Material Requirements Planning (MRP) systems. MRP allowed manufacturers to plan production schedules and raw material purchases based on demand forecasts, transforming what had

previously been a manual calculation exercise into something a computer could handle automatically. Over subsequent decades, MRP evolved into Manufacturing Resource Planning (MRP II) and then into the broader Enterprise Resource Planning (ERP) systems that large corporations use today. These platforms — products like SAP ERP, Oracle NetSuite, and Microsoft Dynamics integrate inventory, procurement, production, finance, and human resources into a single unified system. They are powerful, but they are also expensive, complex, and designed for organisations with dedicated IT departments. For a small distribution business in Osun State, an SAP implementation is not a realistic option.

The emergence of cloud computing and the SaaS model in the 2000s began to change this. Platforms like QuickBooks Commerce (formerly TradeGecko), Zoho Inventory, and Odoo made inventory management software accessible to small businesses through a browser, without requiring the purchase of server hardware or the expertise to manage it. These tools handle stock tracking, purchase orders, sales orders, and basic reporting, and many integrate with popular e-commerce platforms and accounting software. For businesses in markets with reliable internet infrastructure and modest subscription fees denominated in stable currencies, they work well.

However, the Nigerian market has characteristics that most of these platforms were not designed around. Internet connectivity, while improving, remains less reliable and more expensive than in the markets these tools were designed for. The subscription fees, when converted to naira, represent a meaningful percentage of a small business owner's monthly revenue. Customer support is typically provided in time zones far from West Africa. And the workflow assumptions built into the software about payment methods, product categorisation, and regulatory requirements often do not match local practice. As a result, a

significant portion of Nigerian SMEs continue to use spreadsheets and manual processes despite the availability of commercial alternatives.

The multi-tenant architecture that this project adopts directly addresses the cost dimension of this problem. As Kavis (2014) explains, multi-tenancy allows the infrastructure costs of running a SaaS platform to be spread across all tenants, reducing the per-tenant cost substantially compared to maintaining separate deployments. Bezemer et al. (2010) studied the engineering work involved in converting an existing single-tenant system to a multi-tenant one and identified the key challenges: tenant identification, data isolation, and performance management. Their findings informed the design decisions in this project, particularly the choice to use a shared database with separate schemas per tenant. This approach provides strong data isolation without the overhead of running a separate database server for each tenant.

The choice of microservices as the architectural style for the platform is similarly grounded in research. Tizzei et al. (2017) demonstrated that microservices and multi-tenancy can be combined productively, achieving an average code reuse rate of 62 percent across tenant deployments in an industrial study. Batista et al. (2022) reported concrete improvements in scalability and integration capability after migrating a monolithic system to a microservices architecture. Newman (2021) provides a comprehensive treatment of microservices design principles, including service decomposition, API contracts, and the role of messaging systems in keeping services decoupled. These sources collectively establish both the value of the approach and the specific design patterns most likely to succeed.

This project sits at the intersection of these two architectural traditions multi-tenancy and microservices and applies them to a practical problem with clear real-world relevance in the Nigerian context.

1.3 Statement of the Problem

Despite growing awareness of digital business tools, a large proportion of small and medium-sized businesses in Nigeria continue to manage their inventory using spreadsheets, paper records, or outdated desktop software. The Small and Medium Enterprises Development Agency of Nigeria (SMEDAN) has noted in successive reports that limited access to affordable business management software is one of the structural barriers to productivity growth among Nigerian SMEs. The problem is not simply one of awareness many business owners are aware that better tools exist. The problem is one of accessibility, fit, and cost.

A closer examination of the current situation reveals several interrelated issues that a well-designed SaaS inventory platform could address:

i. The majority of commercially available SaaS inventory platforms are priced for markets in Europe and North America. Their subscription fees, when converted to naira at the prevailing exchange rate, are unaffordable for a small trader managing a modest product range. Tools like Cin7, DEAR Inventory, or Linnworks charge fees that represent a significant proportion of a small Nigerian business's monthly revenue, making adoption economically irrational for many potential users.

ii. Single-tenant deployment models impose disproportionate infrastructure costs on vendors serving the low-end market. Under a single-tenant model, each new customer requires its own server environment, database instance, and maintenance overhead. A vendor trying to serve five hundred small Nigerian businesses on a single-tenant model would need to operate five hundred separate deployments, making the unit economics of offering an affordable subscription price extremely difficult.

iii. Most existing inventory platforms are built as monolithic applications. When the vendor needs to release an update, fix a bug, or apply a security patch, the entire application must be restarted or redeployed, briefly interrupting service for all users. In a platform serving many businesses simultaneously, planned downtime is a real operational problem. Staff who depend on real-time stock data during trading hours cannot afford periods when the system is unavailable.

iv. There is a lack of documented implementations of a complete multi-tenant, microservices-based inventory system designed with the Nigerian or broader West African SME context in mind. The existing academic literature Tizzei et al. (2017), Batista et al. (2022), Bezemer et al. (2010) establishes the architectural patterns and their benefits, but does so in the context of large industrial systems or theoretical frameworks rather than practical implementations aimed at the specific affordability and workflow constraints of African small businesses.

These four problems together define the gap that this project addresses. The goal is to design and implement a system that combines multi-tenancy, microservices architecture, and tenant-aware data isolation in a working platform, demonstrating that these approaches can be applied effectively at a scale and price point relevant to Nigerian SMEs.

1.4 Aim and Objectives of the Study

Aim: The aim of this project is to design and implement a multi-tenant Software-as-a-Service (SaaS) platform for business inventory management, built on a microservices architecture, that allows multiple independent businesses to manage stock, sales, and reorder activity securely from a single shared system.

In order to achieve this aim, the following specific objectives have been set:

v. To review existing literature on multi-tenant SaaS design, microservices architecture, and inventory management systems, with the aim of establishing sound and evidence-based design principles for the platform.

vi. To design a tenant-aware microservices architecture for the system, covering four core services: Authentication and Tenant Management, Inventory, Order and Sales, and Notification each with clearly defined responsibilities and communication interfaces.

vii. To implement the core microservices with isolated tenant data, using a shared-database, separate-schema-per-tenant strategy enforced consistently at the data access layer of each service.

viii. To evaluate the implemented system against three defined metrics: API response time under simulated load, the number of concurrent tenants the platform can support within acceptable performance bounds, and data isolation integrity and to report the results.

1.5 Significance of the Study

The significance of this project extends across several groups of stakeholders and several dimensions of value.

For Nigerian small and medium-sized businesses. The most direct beneficiary of a working multi-tenant SaaS inventory platform is the business owner or stock manager who currently relies on spreadsheets or paper records. The platform reduces the per-business cost of adopting cloud-based inventory management, because the infrastructure costs are distributed across all tenants rather than borne by each individually. A shop owner in Ede or a distributor in Osogbo who previously found commercial inventory tools unaffordable becomes a realistic user. Real-time stock visibility, automated low-stock alerts, and sales summaries features that large businesses take for granted become accessible to businesses at the bottom of the market.

For the Nigerian technology sector. The project demonstrates the practical application of cloud-native architectural patterns, including microservices, containerisation, REST-based service interfaces, and asynchronous messaging, in a locally relevant context. It provides a reference implementation that other developers, student researchers, and technology companies can study, evaluate, and adapt. The documented architecture, source code, and evaluation results contribute to an evidence base for how modern software engineering practices can be applied in the Nigerian market, where such documentation remains relatively scarce.

For academic knowledge in computing. The project offers an opportunity to test, in a working system, the architectural claims made in the literature. Tizzei et al. (2017), Batista et al. (2022), and Bezemer et al. (2010) each studied existing systems or proposed frameworks;

they did not build a multi-tenant microservices system from scratch for a defined use case and evaluate it against specific metrics. This project does. The findings particularly around data isolation under the separate-schema approach, and response time behaviour as tenant count increases add a concrete data point to an architectural discussion that has largely been conducted at the level of case studies and theoretical proposals.

For the student researcher. Completing this project builds practical competence across a range of areas that Nigerian employers in the technology industry increasingly require: RESTful API design within individual services, service decomposition, PostgreSQL schema management, Docker containerisation, React.js front-end development, and load testing. These are skills rarely developed to this depth in a classroom setting, and the project provides a practical context in which to develop and demonstrate them.

1.6 Scope of the Study

This project is concerned with the design, implementation, and testing of the core components of a multi-tenant SaaS inventory management platform built on a microservices architecture. The following are within the scope of the work:

- The design of the system architecture, including the React.js tenant client, the four microservices, the PostgreSQL data layer, and the RabbitMQ messaging layer.
- The implementation of four core microservices: the Authentication and Tenant Management Service, the Inventory Service, the Order and Sales Service, and the Notification Service, each built with Node.js and the NestJS framework and containerised with Docker.

- The implementation of tenant data isolation using a shared-database, separate-schema-per-tenant strategy, enforced at the data access layer of each service.
- The implementation of secure REST interfaces within the individual microservices, with each service validating authenticated requests and enforcing tenant data isolation.
- A React.js web front-end providing the inventory dashboard, stock entry forms, sales log, and basic reporting views for authenticated tenant users.
- Payment gateway integration to handle financial transactions, billing and subscription management the platform
- The evaluation of the system against three metrics: API response time under load, scalability as tenant count increases, and data isolation integrity across tenant boundaries.

The following areas fall outside the scope of this project and are identified for potential future development:

- Mobile application development for iOS or Android platforms. The current project provides a responsive web interface, but native mobile applications are not part of this scope.
- Advanced analytics, machine learning-based demand forecasting, or business intelligence dashboards beyond basic stock and sales summaries.

- Commercial deployment of the platform to paying customers. The system will be built, tested, and demonstrated in a development and test environment; production-level deployment is beyond the current academic scope.

1.7 Limitations of the Study

A number of practical constraints limit the scope or generalisability of this project, and they are acknowledged here.

Time constraints. The project must be completed within the period allocated for final-year research. This time pressure means that certain features that would be expected in a commercial product multi-currency support, role-based permissions with fine-grained access control, comprehensive audit logging, and a full suite of integration tests cannot be implemented to production standard and have been deferred. The core architecture and the four defined objectives remain achievable within the available time.

Test environment limitations. Performance and scalability testing will be conducted in a controlled test environment using simulated workloads generated by a load-testing tool, rather than with real tenant traffic. The results will give a reliable indication of how the system behaves under load, but they should not be read as precise benchmarks for a fully production-configured deployment on dedicated cloud infrastructure.

Sample size for qualitative data. The structured interviews conducted during the requirements phase involved a small number of stock managers and small-business owners in the Ede and Osogbo area of Osun State. Their input was valuable for identifying user requirements and shaping the feature set, but the sample is too small to be considered

statistically representative of all Nigerian SMEs. The qualitative findings should be read as directional guidance rather than rigorous generalisations.

Cloud infrastructure costs. Extended testing on commercial cloud platforms incurs costs. Testing will therefore be scoped to what is achievable within free-tier allowances and student credit programmes offered by providers such as Amazon Web Services and Google Cloud Platform. This constrains the scale of the load tests that can be run but does not prevent meaningful evaluation of the system's behaviour.

Scope of multi-tenancy model. This project implements the shared-database, separate-schema model of multi-tenancy, which is one of three common approaches, the others being shared schema and separate databases. The chosen approach balances isolation and cost well for the intended scale, but it may not be the optimal choice for every deployment context.

1.8 Definition of Terms

The following technical terms appear frequently throughout this work. Each is defined below in the context in which it is used in this project.

Multi-tenancy: An architectural principle in which a single running instance of a software application serves multiple customer organisations (tenants) simultaneously, with each tenant's data and configuration kept logically separate from all others, even though they share the same underlying infrastructure and application code (Kavis, 2014).

Tenant: An individual organisation or business that has registered to use the multi-tenant platform. In this project, each tenant represents a separate business, for example, a retail

shop, a wholesale distributor, or a pharmacy that uses the platform to manage its own inventory. Each tenant has its own isolated data environment within the shared system.

SaaS (Software-as-a-Service): A software delivery model in which the application is hosted and maintained by the service provider and made available to users over the internet, typically through a web browser, on a subscription or pay-per-use basis. Users do not install or manage the software themselves.

Microservices Architecture: A software design approach that structures an application as a collection of small, independently deployable services, each responsible for a specific, bounded area of business functionality and communicating with other services through well-defined APIs (Newman, 2021). In contrast to a monolithic application, each microservice can be developed, deployed, and scaled on its own.

API (Application Programming Interface): A defined set of rules and protocols through which one software component exposes its functionality for other components to use. In this project, each microservice exposes REST endpoints that the React.js client and authorised services use to interact with it.

REST (Representational State Transfer): An architectural style for designing networked applications, in which clients and servers communicate over HTTP using a standard set of operations. RESTful APIs use HTTP methods such as GET, POST, PUT, and DELETE to represent actions on resources identified by URLs.

Microservice: One of the independently deployable services that together make up the platform. In this project there are four core microservices: the Authentication and Tenant

Management Service, the Inventory Service, the Order and Sales Service, and the Notification Service.

Schema (Database): A logical namespace within a relational database that contains a set of tables, views, and other objects. In PostgreSQL, multiple schemas can exist within a single database instance. This project uses a separate schema for each tenant, providing data isolation without the overhead of a separate database server per tenant.

PostgreSQL: An open-source, object-relational database management system known for its standards compliance, robustness, and support for advanced features including schema-level namespace management. It is the database engine used in this project.

Docker: An open-source platform for developing, shipping, and running applications inside containers. A container packages a service together with all its runtime dependencies into a portable, standardised unit that runs consistently regardless of the host environment.

Container: A lightweight, isolated execution environment produced by Docker. Each microservice in this project is deployed as a container, allowing it to be started, stopped, and scaled independently of the other services.

JWT (JSON Web Token): A compact, URL-safe format for representing claims between two parties. In this project, the Authentication and Tenant Management Service issues a JWT to each user upon login. The token carries the user's identity and tenant identifier, and each protected microservice validates the token before performing a tenant-scoped operation.

RabbitMQ: An open-source message broker that enables services to communicate asynchronously. When one service needs to notify another of an event for example, when

stock falls below the reorder threshold it publishes a message to a RabbitMQ queue. The receiving service consumes the message at its own pace, without either service needing to call the other directly.

NestJS: A Node.js framework for building server-side applications, written in TypeScript. NestJS provides a modular architecture with built-in support for dependency injection, which makes it well suited to building microservices. It is used in this project as the backend framework for each of the four microservices.

React.js: A JavaScript library developed by Meta (Facebook) for building user interfaces. React uses a component-based model in which the interface is composed of reusable, stateful components that update efficiently when the underlying data changes. It is used in this project to build the tenant-facing inventory dashboard.

Scalability: The ability of a system to handle an increasing amount of work by adding resources. In the context of this project, scalability refers specifically to the platform's ability to serve additional tenants and additional concurrent users within each tenant's account without a significant degradation in response time or reliability.

Data Isolation: The guarantee that one tenant's data cannot be read or modified by another tenant, whether through normal application use or through attempts to exploit access control weaknesses. Data isolation is one of the core requirements of any multi-tenant system and is the primary criterion against which the database design in this project is evaluated.

Monolith / Monolithic Application: A software application built as a single, tightly coupled unit in which all components user interface, business logic, and data access are packaged and

deployed together. Any change to any part of the application requires the entire application to be rebuilt and redeployed.

Load Testing: A method of software testing in which the system is subjected to a defined level of simulated user load to measure its performance characteristics, typically response time, throughput, and error rate under that load. In this project, load testing is used to evaluate response time and scalability.

1.9 Organisation of the Study

This project report is organised into five chapters, each addressing a distinct phase of the research and development process.

Chapter One the current chapter, introduces the project. It establishes the context and motivation for the work, identifies the specific problems being addressed, states the aim and objectives, defines the scope and limitations, and provides definitions for the key technical terms used throughout the report.

Chapter Two presents a review of related literature. It surveys published research on multi-tenant SaaS architecture, microservices design, and inventory management systems. The chapter evaluates the strengths and limitations of the existing work and identifies the gap that this project is positioned to fill.

Chapter Three describes the methodology adopted in this project. It covers the research approach and data collection methods used, the design of the system architecture in detail, the rationale for the choice of development tools and frameworks, and the Agile (Scrum) development model under which the implementation was conducted.

Chapter Four presents the implementation and testing of the system. It documents the construction of the four microservices, the implementation of REST interfaces and tenant data isolation, the development of the web front-end, and the results of the three defined evaluation tests: response time under load, scalability, and data isolation integrity.

Chapter Five concludes the report. It summarises the key findings, reflects on the extent to which each of the four objectives was achieved, identifies the contributions of the project to knowledge, and makes recommendations for future work and development of the platform.

References

Batista, C., Proença, B., Cavalcante, E., Batista, T., Morais, F., & Medeiros, H. (2022). Towards a multi-tenant microservice architecture: An industrial experience. In Proceedings of the 46th IEEE Annual Computers, Software, and Applications Conference (pp. 553–562). IEEE.

Batista, C., Cavalcante, W. B. R., Proença, B., Batista, T., & Morais, F. (2024). Managing asynchronous workloads in a multi-tenant microservice enterprise environment. Software: Practice and Experience, 54(2), 334–359.

Bezemer, C.-P., Zaidman, A., Platzbeecker, B., Hurkmans, T., & Hart, A. (2010). Enabling multi-tenancy: An industrial experience report. In Proceedings of the 2010 IEEE International Conference on Software Maintenance (pp. 1–8). IEEE.

Kavis, M. J. (2014). Architecting the cloud: Design decisions for cloud computing service models (SaaS, PaaS, and IaaS). John Wiley & Sons.

Mietzner, R., Metzger, A., Leymann, F., & Pohl, K. (2009). Variability modeling to support customization and deployment of multi-tenant-aware software as a service applications. In Proceedings of the ICSE Workshop on Principles of Engineering Service-Oriented Systems (pp. 18–25). IEEE Computer Society.

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

Tizzei, L. P., Nery, M., Segura, V. C. V. B., & Cerqueira, R. F. G. (2017). Using microservices and software product line engineering to support reuse of evolving multi-tenant SaaS. In Proceedings of the 21st International Systems and Software Product Line Conference – Volume A (pp. 205–214). ACM.